

OAIBOX CSI Data Extraction Documentation for ISAC 6G

1. Setting up the OAIBOX for CSI Data Reporting

In order to set up the OAIBOX for CSI Data Reporting, the following changes were made:

Step 1:

In the directory `/home/user/openairinterface5g/openair1/PHY/NR_ESTIMATION` create the file `custom_time_functions.c` with the content below, in order to allow the creation of log files with timestamps.

```
#include <bits/time.h>
#include <stdio.h>
#include <time.h>

long long current_time_sec()
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    return ts.tv_sec;
}

long long current_time_ms()
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    return ts.tv_sec * 1000LL + ts.tv_nsec / 1000000LL;
}

long long current_time_us()
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    return ts.tv_sec * 1000000LL + ts.tv_nsec / 1000LL;
}

void print_formatted_time(FILE* fptr)
```

```

{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);

    struct tm tm_info;
    localtime_r(&ts.tv_sec, &tm_info);

    char buffer[20];
    strftime(buffer, sizeof(buffer), "%y:%m:%d:%H:%M:%S", &tm_info);
    fprintf(fp, "Timestamp: %s:%06ld\n", buffer, ts.tv_nsec / 1000);
    // format:   YY:MM:DD:hh:mm:ss:us
}

void formatted_time_string(char* string, short len)
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);

    struct tm tm_info;
    localtime_r(&ts.tv_sec, &tm_info);

    strftime(string, len, "%y_%m_%d_%H_%M", &tm_info);
}

FILE* get_new_logfile()
{
    char filename[40], formatted_time[20];
    formatted_time_string(formatted_time, 20);
    sprintf(filename, "srs_debug%s.log", formatted_time);
    return fopen(filename, "a");
}

```

Step 2:

In the file

/home/user/openairinterface5g/openair1/PHY/NR_ESTIMATION/nr_ul_channel_estimation.c

add the following headers at line 7, as well as the 3 global variables below:

```

#include <stdio.h>
#include <time.h>
#include "custom_time_functions_utils.c"

FILE* srs_debug_fptr = NULL;

```

```
int first_time_flag = 1;
long long previousTimestamp;
```

In the same file, at the beginning of the function `nr_srs_channel_estimation()`, just under the second apparition of the macro `SRS_DEBUG` (around line 723), add the following code block, commenting the existing code in the preprocessor `ifdef-endif` block (like below):

```
if (srs_debug_fptr == NULL && first_time_flag) {
    srs_debug_fptr = get_new_logfile();
    previousTimestamp = current_time_sec();
    first_time_flag = 0;
    LOG_I(NR_PHY, "Opening srs_debug.log for the first time.");
} else if (srs_debug_fptr != NULL && !first_time_flag) {
    if (current_time_sec() - previousTimestamp > 3600) {
        fclose(srs_debug_fptr);
        srs_debug_fptr = get_new_logfile();
        previousTimestamp = current_time_sec();
        LOG_I(NR_PHY, "Opening new srs_debug.log file.");
    }
} else {
    if (srs_debug_fptr != NULL)
        fclose(srs_debug_fptr);
    LOG_I(NR_PHY, "ERROR: Couldn't open file srs_debug.log");
}

if (srs_debug_fptr != NULL) {
    print_formatted_time(srs_debug_fptr);
    fprintf(srs_debug_fptr, "Calling %s function\n", __FUNCTION__);
}
// LOG_I(NR_PHY, "Calling %s function\n", __FUNCTION__);
```

At the 3rd apparition of the macro `SRS_DEBUG` (around line 788), add the following code block, commenting the existing code in the preprocessor `ifdef-endif` block (like below):

```
// LOG_I(NR_PHY, "===== UE port %d --> gNB Rx antenna %i
===== \n", p_index, ant);
if (srs_debug_fptr != NULL) {
    print_formatted_time(srs_debug_fptr);
    fprintf(srs_debug_fptr, "===== UE port %d --> gNB Rx antenna %i
===== \n", p_index, ant);
}
```

At the 4th apparition of the macro SRS_DEBUG (around line 833), add the following code block, commenting the existing code in the preprocessor ifdef-endif block:

```

/*
int subcarrier_log = subcarrier - subcarrier_offset;
if (subcarrier_log < 0) {
    subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
}
if (subcarrier_log % 12 == 0) {
    LOG_I(NR_PHY, "----- %d
-----\n", subcarrier_log / 12);

    LOG_I(NR_PHY, "\t
__genRe__ __genIm__ | __rxRe__ __rxIm__ | __lsRe__ __lsIm__\n");
}
LOG_I(NR_PHY,
    "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
    subcarrier_log,
    ((c16_t *)srs_generated_signal[p_index])[subcarrier].r,
    ((c16_t *)srs_generated_signal[p_index])[subcarrier].i,
    ((c16_t *)srs_received_signal[ant])[subcarrier].r,
    ((c16_t *)srs_received_signal[ant])[subcarrier].i,
    ls_estimated.r,
    ls_estimated.i);
*/

if (srs_debug_fptr != NULL) {
    int subcarrier_log = subcarrier - subcarrier_offset;
    if (subcarrier_log < 0) {
        subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
    }
    if (subcarrier_log % 12 == 0) {
        fprintf(srs_debug_fptr,
            "----- %d
-----\n",
            subcarrier_log / 12);

        fprintf(srs_debug_fptr, "\t
__genRe__ __genIm__ | __rxRe__ __rxIm__ | __lsRe__ __lsIm__\n");
    }
    fprintf(srs_debug_fptr,
        "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
        subcarrier_log,
        ((c16_t *)srs_generated_signal[p_index])[subcarrier].r,
        ((c16_t *)srs_generated_signal[p_index])[subcarrier].i,
        ((c16_t *)srs_received_signal[ant])[subcarrier].r,
        ((c16_t *)srs_received_signal[ant])[subcarrier].i,

```

```

        ls_estimated.r,
        ls_estimated.i);
}

```

At the 5th apparition of the macro SRS_DEBUG (around line 950), add the following code block, commenting the existing code in the ifdef-endif preprocessor block (like below):

```

/*
subcarrier = subcarrier_offset + nr_srs_info->k_0_p[p_index][0];
if (subcarrier > frame_parms->ofdm_symbol_size) {
    subcarrier -= frame_parms->ofdm_symbol_size;
}

for (int k = 0; k < K_TC * M_sc_b_SRS; k++) {
    int subcarrier_log = subcarrier - subcarrier_offset;
    if (subcarrier_log < 0) {
        subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
    }

    if (subcarrier_log % 12 == 0) {
        LOG_I(NR_PHY, "----- %d
-----\n", subcarrier_log / 12);
        LOG_I(NR_PHY, "\t
__lsRe__      __lsIm__ | __intRe__  __intIm__ | __noiRe__  __noiIm__ \n");
    }

    LOG_I(NR_PHY,
        "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
        subcarrier_log,
        srs_ls_estimated_channel[subcarrier].r,
        srs_ls_estimated_channel[subcarrier].i,
        ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier].r,
        ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier].i,
        noise[base_idx + (k / K_TC)].r,
        noise[base_idx + (k / K_TC)].i);

    // Subcarrier increment
    subcarrier++;
    if (subcarrier >= frame_parms->ofdm_symbol_size) {
        subcarrier = subcarrier - frame_parms->ofdm_symbol_size;
    }
}
*/

```

```

if (srs_debug_fptr != NULL) {
    subcarrier = subcarrier_offset + nr_srs_info->k_0_p[p_index][0];
    if (subcarrier > frame_parms->ofdm_symbol_size) {
        subcarrier -= frame_parms->ofdm_symbol_size;
    }

    for (int k = 0; k < K_TC * M_sc_b_SRS; k++) {
        int subcarrier_log = subcarrier - subcarrier_offset;
        if (subcarrier_log < 0) {
            subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
        }

        if (subcarrier_log % 12 == 0) {
            fprintf(srs_debug_fptr,
                    "----- %d\n",
                    subcarrier_log / 12);
            fprintf(srs_debug_fptr, "\t
__lsRe__      __lsIm__ | __intRe__      __intIm__ | __noiRe__      __noiIm__\n");

        }

        fprintf(srs_debug_fptr,
                "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
                subcarrier_log,
                srs_ls_estimated_channel[subcarrier].r,
                srs_ls_estimated_channel[subcarrier].i,
                ((c16_t
*)srs_estimated_channel_freq[ant][p_index])[subcarrier].r,
                ((c16_t
*)srs_estimated_channel_freq[ant][p_index])[subcarrier].i,
                noise[base_idx + (k / K_TC)].r,
                noise[base_idx + (k / K_TC)].i);

        // Subcarrier increment
        subcarrier++;
        if (subcarrier >= frame_parms->ofdm_symbol_size) {
            subcarrier = subcarrier - frame_parms->ofdm_symbol_size;
        }
    }
}

```

At the 6th apparition of the macro SRS_DEBUG (around line 1043), add the following code block, commenting the existing code in the preprocessor ifdef-endif block:

```

// LOG_I(NR_PHY, "signal_power = %u\n", signal_power);

if (srs_debug_fptr != NULL) {
    fprintf(srs_debug_fptr, "signal_power = %u\n", signal_power);
}

```

At the 7th apparition of the macro SRS_DEBUG (around line 1087), add the following code block, commenting the existing code in the preprocessor ifdef-endif block:

```

// LOG_I(NR_PHY, "noise_power_per_rb[%i] = %i, snr_per_rb[%i] = %i dB\n", rb,
noise_power_per_rb[rb], rb, snr_per_rb[rb]);

if (srs_debug_fptr != NULL) {
    fprintf(srs_debug_fptr,
        "noise_power_per_rb[%i] = %i, snr_per_rb[%i] = %i dB\n",
        rb,
        noise_power_per_rb[rb],
        rb,
        snr_per_rb[rb]);
}

```

At the 8th apparition of the macro SRS_DEBUG (around line 1106), add the following code block, commenting the existing code in the preprocessor ifdef-endif block:

```

// LOG_I(NR_PHY, "noise_power = %u, SNR = %i dB\n", noise_power, *snr);

if (srs_debug_fptr != NULL) {
    fprintf(srs_debug_fptr, "noise_power = %u, SNR = %i dB\n", noise_power, *snr);
}

```

In the end, the file nr_ul_channel_estimation.c should look like down below.

```

/*
 * Licensed to the OpenAirInterface (OAI) Software Alliance under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The OpenAirInterface Software Alliance licenses this file to You under
 * the OAI Public License, Version 1.0 (the "License"); you may not use this file
 * except in compliance with the License.
 * You may obtain a copy of the License at
 *

```

[illegible]


```

int localOffset1 = *offset1;
int localOffset2 = *offset2;
c32_t cumul = {0};
for (int i = 0; i < N; i++) {
    cumul = c32x16maddShift(in1[localOffset1], in2[localOffset2], cumul, 15);
    localOffset1 += step1;
    localOffset2 = (localOffset2 + step2) % modulo2;
}
*offset1 = localOffset1;
*offset2 = localOffset2;
return c16x32div(cumul, N);
}

static void nr_pusch_antenna_processing(void *arg)
{
    puschAntennaProc_t *rdata = (puschAntennaProc_t *)arg;
    unsigned char Ns = rdata->Ns;
    int nl = rdata->nl;
    unsigned short p = rdata->p;
    unsigned char symbol = rdata->symbol;
    int aarx = rdata->aarx;
    int numAntennas = rdata->numAntennas;
    unsigned short bwp_start_subcarrier = rdata->bwp_start_subcarrier;
    nfapi_nr_pusch_pdu_t *pusch_pdu = rdata->pusch_pdu;
    int *max_ch = rdata->max_ch;
    c16_t *pilot = rdata->pilot;
    uint64_t noise_amp2 = *(rdata->noise_amp2);
    int nest_count = *(rdata->nest_count);
    delay_t *delay = rdata->delay;

    const int chest_freq = rdata->chest_freq;
    NR_gNB_PUSCH *pusch_vars = rdata->pusch_vars;
    c16_t **ul_ch_estimates = (c16_t **)pusch_vars->ul_ch_estimates;
    NR_DL_FRAME_PARMS *frame_parms = rdata->frame_parms;
    const int symbolSize = frame_parms->ofdm_symbol_size;
    const int slot_offset = (Ns & 3) * frame_parms->symbols_per_slot * symbolSize;
    const int delta = get_delta(p, pusch_pdu->dmrs_config_type);
    const int symbol_offset = symbolSize * symbol;
    const int k0 = bwp_start_subcarrier;
    const int nb_rb_pusch = pusch_pdu->rb_size;
    const int beam_nb = rdata->beam_nb;
    for (int antenna = aarx; antenna < aarx + numAntennas; antenna++) {
        c16_t ul_ls_est[symbolSize] __attribute__((aligned(32)));
        memset(ul_ls_est, 0, sizeof(c16_t) * symbolSize);
        c16_t *rxdataF = (c16_t *)&rdata->rxdataF[beam_nb][antenna][symbol_offset +
slot_offset];
        c16_t *ul_ch = &ul_ch_estimates[nl * frame_parms->nb_antennas_rx +
antenna][symbol_offset];
        memset(ul_ch, 0, sizeof(*ul_ch) * symbolSize);

        LOG_D(PHY,
            "symbol_offset %d, slot_offset %d, OFDM size %d, Ns = %d, k0 = %d, symbol %d\n",

```

```

        symbol_offset,
        slot_offset,
        symbolSize,
        Ns,
        k0,
        symbol);

#ifdef DEBUG_PUSCH
LOG_I(PHY, "symbol_offset %d, delta %d\n", symbol_offset, delta);
LOG_I(PHY, "ch est pilot, N_RB_UL %d\n", frame_parms->N_RB_UL);
LOG_I(PHY,
        "bwp_start_subcarrier %d, k0 %d, first_carrier %d, nb_rb_pusch %d\n",
        bwp_start_subcarrier,
        k0,
        frame_parms->first_carrier_offset,
        nb_rb_pusch);
LOG_I(PHY, "ul_ch addr %p \n", ul_ch);
#endif

if (pusch_pdu->dmrs_config_type == pusch_dmrs_type1 && chest_freq == 0) {
    c16_t *pil = pilot;
    int re_offset = k0;
    LOG_D(PHY, "PUSCH estimation DMRS type 1, Freq-domain interpolation");
    int pilot_cnt = 0;

    for (int n = 0; n < 3 * nb_rb_pusch; n++) {
        // LS estimation
        c32_t ch = {0};

        for (int k_line = 0; k_line <= 1; k_line++) {
            re_offset = (k0 + (n << 2) + (k_line << 1) + delta) % symbolSize;
            ch = c32x16maddShift(*pil, rxdataF[re_offset], ch, 16);
            pil++;
        }

        c16_t ch16 = {.r = (int16_t)ch.r, .i = (int16_t)ch.i};
        *max_ch = max(abs(ch.r), abs(ch.i));
        for (int k = pilot_cnt << 1; k < (pilot_cnt << 1) + 4; k++) {
            ul_ls_est[k] = ch16;
        }
        pilot_cnt += 2;
    }

    nr_est_delay(frame_parms->ofdm_symbol_size, ul_ls_est, (c16_t
*)pusch_vars->ul_ch_estimates_time[antenna], delay);
    int delay_idx = get_delay_idx(delay->est_delay, MAX_DELAY_COMP);
    c16_t *ul_delay_table = frame_parms->delay_table[delay_idx];

#ifdef DEBUG_PUSCH
    printf("Estimated delay = %i\n", delay->est_delay >> 1);
#endif
}

```

```

pilot_cnt = 0;
for (int n = 0; n < 3 * nb_rb_pusch; n++) {
    // Channel interpolation
    for (int k_line = 0; k_line <= 1; k_line++) {
        // Apply delay
        int k = pilot_cnt << 1;
        c16_t ch16 = c16mulShift(ul_ls_est[k], ul_delay_table[k], 8);

#ifdef DEBUG_PUSCH
        re_offset = (k0 + (n << 2) + (k_line << 1)) % symbolSize;
        c16_t *rxF = &rxdataF[re_offset];
        printf("pilot %4d: pil -> (%6d,%6d), rxF -> (%4d,%4d), ch -> (%4d,%4d)\n",
            pilot_cnt,
            pil->r,
            pil->i,
            rxF->r,
            rxF->i,
            ch.r,
            ch.i);
#endif

        if (pilot_cnt == 0) {
            c16multaddVectRealComplex(filt16_ul_p0, &ch16, ul_ch, 16);
        } else if (pilot_cnt == 1 || pilot_cnt == 2) {
            c16multaddVectRealComplex(filt16_ul_p1p2, &ch16, ul_ch, 16);
        } else if (pilot_cnt == (6 * nb_rb_pusch - 1)) {
            c16multaddVectRealComplex(filt16_ul_last, &ch16, ul_ch, 16);
        } else {
            c16multaddVectRealComplex(filt16_ul_middle, &ch16, ul_ch, 16);
            if (pilot_cnt % 2 == 0) {
                ul_ch += 4;
            }
        }

        pilot_cnt++;
    }
}

// Revert delay
pilot_cnt = 0;
ul_ch = &ul_ch_estimates[nl * frame_params->nb_antennas_rx + antenna][symbol_offset];
int inv_delay_idx = get_delay_idx(-delay->est_delay, MAX_DELAY_COMP);
c16_t *ul_inv_delay_table = frame_params->delay_table[inv_delay_idx];
for (int n = 0; n < 3 * nb_rb_pusch; n++) {
    for (int k_line = 0; k_line <= 1; k_line++) {
        int k = pilot_cnt << 1;
        ul_ch[k] = c16mulShift(ul_ch[k], ul_inv_delay_table[k], 8);
        ul_ch[k + 1] = c16mulShift(ul_ch[k + 1], ul_inv_delay_table[k + 1], 8);
        noise_amp2 += c16amp2(c16sub(ul_ls_est[k], ul_ch[k]));
        noise_amp2 += c16amp2(c16sub(ul_ls_est[k + 1], ul_ch[k + 1]));
    }
}

#ifdef DEBUG_PUSCH

```

```

        re_offset = (k0 + (n << 2) + (k_line << 1)) % symbolSize;
        printf("ch -> (%4d,%4d), ch_inter -> (%4d,%4d)\n", ul_ls_est[k].r, ul_ls_est[k].i,
ul_ch[k].r, ul_ch[k].i);
    #endif

        pilot_cnt++;
        nest_count += 2;
    }
}

} else if (pusch_pdu->dmrs_config_type == pusch_dmrs_type2
    && chest_freq == 0) { // pusch_dmrs_type2 |p_r,p_l,d,d,d,d,p_r,p_l,d,d,d,d|
    LOG_D(PHY, "PUSCH estimation DMRS type 2, Freq-domain interpolation\n");
    c16_t *pil = pilot;
    c16_t *rx = &rxdataF[delta];
    for (int n = 0; n < nb_rb_pusch * NR_NB_SC_PER_RB; n += 6) {
        c16_t ch0 = c16mulShift(*pil, rx[(k0 + n) % symbolSize], 15);
        pil++;
        c16_t ch1 = c16mulShift(*pil, rx[(k0 + n + 1) % symbolSize], 15);
        pil++;
        c16_t ch = c16addShift(ch0, ch1, 1);
        *max_ch = max(abs(ch.r), abs(ch.i));
        multadd_real_four_symbols_vector_complex_scalar(filt8_rep4, &ch, &ul_ls_est[n]);
        ul_ls_est[n + 4] = ch;
        ul_ls_est[n + 5] = ch;
        noise_amp2 += c16amp2(c16sub(ch0, ch));
        nest_count += 1;
    }

    // Delay compensation
    nr_est_delay(frame_parms->ofdm_symbol_size, ul_ls_est, (c16_t
*)pusch_vars->ul_ch_estimates_time[antenna], delay);
    int delay_idx = get_delay_idx(-delay->est_delay, MAX_DELAY_COMP);
    c16_t *ul_delay_table = frame_parms->delay_table[delay_idx];
    for (int n = 0; n < nb_rb_pusch * NR_NB_SC_PER_RB; n++) {
        ul_ch[n] = c16mulShift(ul_ls_est[n], ul_delay_table[n % 6], 8);
    }

}

// this is case without frequency-domain linear interpolation, just take average of LS
channel estimates of 6 DMRS REs and use a
// common value for the whole PRB
else if (pusch_pdu->dmrs_config_type == pusch_dmrs_type1) {
    LOG_D(PHY, "PUSCH estimation DMRS type 1, no Freq-domain interpolation\n");
    c16_t *rxF = &rxdataF[delta];
    int pil_offset = 0;
    int re_offset = k0;
    c16_t ch;

    // First PRB
    ch = c32x16cumulVectVectWithSteps(pilot, &pil_offset, 1, rxF, &re_offset, 2,
symbolSize, 6);

```

```

#if NO_INTERP
    for (c16_t *end = ul_ch + 12; ul_ch < end; ul_ch++)
        *ul_ch = ch;
#else
    c16multaddVectRealComplex(filt8_avlip0, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip1, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip2, &ch, ul_ch, 8);
    ul_ch -= 12;
#endif

    for (int pilot_cnt = 6; pilot_cnt < 6 * (nb_rb_pusch - 1); pilot_cnt += 6) {
        ch = c32x16cumulVectVectWithSteps(pilot, &pil_offset, 1, rxF, &re_offset, 2,
symbolSize, 6);
        *max_ch = max(abs(ch.r), abs(ch.i));
    }

#if NO_INTERP
    for (c16_t *end = ul_ch + 12; ul_ch < end; ul_ch++)
        *ul_ch = ch;
#else
    ul_ch[3].r += (ch.r * 1365) >> 15; // 1/12*16384
    ul_ch[3].i += (ch.i * 1365) >> 15; // 1/12*16384

    ul_ch += 4;
    c16multaddVectRealComplex(filt8_avlip3, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip4, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip5, &ch, ul_ch, 8);
    ul_ch -= 8;
#endif
}
// Last PRB
ch = c32x16cumulVectVectWithSteps(pilot, &pil_offset, 1, rxF, &re_offset, 2,
symbolSize, 6);

#if NO_INTERP
    for (c16_t *end = ul_ch + 12; ul_ch < end; ul_ch++)
        *ul_ch = ch;
#else
    ul_ch[3].r += (ch.r * 1365) >> 15; // 1/12*16384
    ul_ch[3].i += (ch.i * 1365) >> 15; // 1/12*16384

    ul_ch += 4;
    c16multaddVectRealComplex(filt8_avlip3, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip6, &ch, ul_ch, 8);
#endif
} else { // this is case without frequency-domain linear interpolation, just take
average of LS channel estimates of 4 DMRS REs
    // and use a common value for the whole PRB

```

```

LOG_D(PHY, "PUSCH estimation DMRS type 2, no Freq-domain interpolation");
c16_t *pil = pilot;
int re_offset = (k0 + delta) % symbolSize;
c32_t ch0 = {0};
// First PRB
ch0 = c32x16mulShift(*pil, rxdataF[re_offset], 15);
pil++;
re_offset = (re_offset + 1) % symbolSize;
ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
pil++;
re_offset = (re_offset + 5) % symbolSize;
ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
re_offset = (re_offset + 1) % symbolSize;
ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
pil++;
re_offset = (re_offset + 5) % symbolSize;

c16_t ch = c16x32div(ch0, 4);
#if NO_INTERP
for (c16_t *end = ul_ch + 12; ul_ch < end; ul_ch++)
    *ul_ch = ch;
#else
c16multaddVectRealComplex(filt8_avlip0, &ch, ul_ch, 8);
ul_ch += 8;
c16multaddVectRealComplex(filt8_avlip1, &ch, ul_ch, 8);
ul_ch += 8;
c16multaddVectRealComplex(filt8_avlip2, &ch, ul_ch, 8);
ul_ch -= 12;
#endif

for (int pilot_cnt = 4; pilot_cnt < 4 * (nb_rb_pusch - 1); pilot_cnt += 4) {
    c32_t ch0;
    ch0 = c32x16mulShift(*pil, rxdataF[re_offset], 15);
    pil++;
    re_offset = (re_offset + 1) % symbolSize;

    ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
    pil++;
    re_offset = (re_offset + 5) % symbolSize;

    ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
    pil++;
    re_offset = (re_offset + 1) % symbolSize;

    ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
    pil++;
    re_offset = (re_offset + 5) % symbolSize;

    ch = c16x32div(ch0, 4);
    *max_ch = max(abs(ch.r), abs(ch.i));
}
#if NO_INTERP

```

```

        for (c16_t *end = ul_ch + 12; ul_ch < end; ul_ch++)
            *ul_ch = ch;
#else
    ul_ch[3] = c16maddShift(ch, (c16_t){1365, 1365}, (c16_t){0, 0}, 15); // 1365 =
1/12*16384 (full range is +/- 32768)
    ul_ch += 4;
    c16multaddVectRealComplex(filt8_avlip3, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip4, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip5, &ch, ul_ch, 8);
    ul_ch -= 8;
#endif
}

// Last PRB
ch0 = c32x16mulShift(*pil, rxdataF[re_offset], 15);
pil++;
re_offset = (re_offset + 1) % symbolSize;

ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
pil++;
re_offset = (re_offset + 5) % symbolSize;

ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
pil++;
re_offset = (re_offset + 1) % symbolSize;

ch0 = c32x16maddShift(*pil, rxdataF[re_offset], ch0, 15);
pil++;
re_offset = (re_offset + 5) % symbolSize;

ch = c16x32div(ch0, 4);
#if NO_INTERP
    for (c16_t *end = ul_ch + 12; ul_ch < end; ul_ch++)
        *ul_ch = ch;
#else
    ul_ch[3] = c16maddShift(ch, (c16_t){1365, 1365}, (c16_t){0, 0}, 15); // 1365 =
1/12*16384 (full range is +/- 32768)
    ul_ch += 4;
    c16multaddVectRealComplex(filt8_avlip3, &ch, ul_ch, 8);
    ul_ch += 8;
    c16multaddVectRealComplex(filt8_avlip6, &ch, ul_ch, 8);
#endif
}

#ifdef DEBUG_PUSCH
    ul_ch = &ul_ch_estimates[nl * gNB->frame_parms.nb_antennas_rx + aarx][symbol_offset];
    for (int idxP = 0; idxP < ceil((float)nb_rb_pusch * 12 / 8); idxP++) {
        for (int idxI = 0; idxI < 8; idxI++) {
            printf("%d\t%d\t", ul_ch[idxP * 8 + idxI].r, ul_ch[idxP * 8 + idxI].i);
        }
    }

```

```

        printf("%d\n", idxP);
    }
#endif
    // update the values inside the arrays
    *(rdata->noise_amp2) = noise_amp2;
    *(rdata->nest_count) = nest_count;
}
completed_task_ans(rdata->ans);
}

int nr_pusch_channel_estimation(PHY_VARS_gNB *gNB,
                                unsigned char Ns,
                                int nl,
                                unsigned short p,
                                unsigned char symbol,
                                int ul_id,
                                int beam_nb,
                                unsigned short bwp_start_subcarrier,
                                nfapi_nr_pusch_pdu_t *pusch_pdu,
                                int *max_ch,
                                uint32_t *nvar)
{
    c16_t pilot[3280] __attribute__((aligned(32)));

#ifdef DEBUG_CH
    FILE *debug_ch_est;
    debug_ch_est = fopen("debug_ch_est.txt", "w");
#endif

    const int nb_rb_pusch = pusch_pdu->rb_size;

    //-----generate DMRS-----//

    if (pusch_pdu->transform_precoding == transformPrecoder_disabled) {
        // Note: pilot returned by the following function is already the complex conjugate of
        the transmitted DMRS
        NR_DL_FRAME_PARMS *fp = &gNB->frame_parms;
        const uint32_t *gold = nr_gold_pusch(fp->N_RB_UL,
                                              fp->symbols_per_slot,
                                              gNB->gNB_config.cell_config.phy_cell_id.value,
                                              pusch_pdu->scid,
                                              Ns,
                                              symbol);

        nr_pusch_dmrs_rx(gNB,
                        Ns,
                        gold,
                        pilot,
                        (1000 + p),
                        0,
                        nb_rb_pusch,
                        (pusch_pdu->bwp_start + pusch_pdu->rb_start) * NR_NB_SC_PER_RB,
                        pusch_pdu->dmrs_config_type);
    }
}

```



```

} else { // if transform precoding or SC-FDMA is enabled in Uplink
// NR_SC_FDMA supports type1 DMRS so only 6 DMRS REs per RB possible
const int index = get_index_for_dmrs_lowpapr_seq(nb_rb_pusch * (NR_NB_SC_PER_RB / 2));
const uint8_t u = pusch_pdu->dfts_ofdm.low_papr_group_number;
const uint8_t v = pusch_pdu->dfts_ofdm.low_papr_sequence_number;
c16_t *dmrs_seq = gNB_dmrs_lowpaprtype1_sequence[u][v][index];
LOG_D(PHY, "Transform Precoding params. u: %d, v: %d, index for dmrsseq: %d\n", u, v,
index);
AssertFatal(index >= 0,
            "Num RBs not configured according to 3GPP 38.211 section 6.3.1.4. For PUSCH
with transform precoding, num RBs "
            "cannot be multiple of any other primenumber other than 2,3,5\n");
AssertFatal(dmrs_seq != NULL, "DMRS low PAPR seq not found, check if DMRS sequences are
generated");
nr_pusch_lowpaprtype1_dmrs_rx(gNB, Ns, dmrs_seq, pilot, 1000, 0, nb_rb_pusch, 0,
pusch_pdu->dmrs_config_type);
#ifdef DEBUG_PUSCH
    printf("NR_UL_CHANNEL_EST: index %d, u %d,v %d\n", index, u, v);
    LOG_M("gNb_DMRS_SEQ.m", "gNb_DMRS_SEQ", dmrs_seq, 6 * nb_rb_pusch, 1, 1);
#endif
}
//-----//

#ifdef DEBUG_PUSCH

    for (int i = 0; i < (6 * nb_rb_pusch); i++) {
        LOG_I(PHY, "In %s: %d + j*(%d)\n", __FUNCTION__, pilot[i].r, pilot[i].i);
    }

#endif

    int nest_count = 0;
    uint64_t noise_amp2 = 0;
    delay_t *delay = &gNB->ulsch[ul_id].delay;
    memset(delay, 0, sizeof(*delay));

    int nb_antennas_rx = gNB->frame_parms.nb_antennas_rx;
    delay_t delay_arr[nb_antennas_rx];
    uint64_t noise_amp2_arr[nb_antennas_rx];
    int max_ch_arr[nb_antennas_rx];
    int nest_count_arr[nb_antennas_rx];

    for (int i = 0; i < nb_antennas_rx; ++i) {
        max_ch_arr[i] = *max_ch;
        nest_count_arr[i] = nest_count;
        noise_amp2_arr[i] = noise_amp2;
        delay_arr[i] = *delay;
    }

    notifiedFIFO_t respPuschAarx;
    initNotifiedFIFO(&respPuschAarx);
    start_meas(&gNB->pusch_channel_estimation_antenna_processing_stats);

```

```

int numAntennas = gNB->dms_num_antennas_per_thread;
int num_jobs = CEILIDIV(gNB->frame_parms.nb_antennas_rx, numAntennas);
puschAntennaProc_t rdatas[num_jobs];
memset(rdatas, 0, sizeof(rdatas));
task_ans_t ans[num_jobs];
memset(ans, 0, sizeof(ans));
for (int job_id = 0; job_id < num_jobs; job_id++) {
    puschAntennaProc_t *rdata = &rdatas[job_id];
    task_t task = {.func = nr_pusch_antenna_processing, .args = rdata};

    // Local init in the current loop
    rdata->Ns = Ns;
    rdata->nl = nl;
    rdata->p = p;
    rdata->symbol = symbol;
    rdata->aarx = job_id * numAntennas;
    rdata->numAntennas = numAntennas;
    rdata->bwp_start_subcarrier = bwp_start_subcarrier;
    rdata->pusch_pdu = pusch_pdu;
    rdata->max_ch = &max_ch_arr[rdata->aarx];
    rdata->pilot = pilot;
    rdata->nest_count = &nest_count_arr[rdata->aarx];
    rdata->noise_amp2 = &noise_amp2_arr[rdata->aarx];
    rdata->delay = &delay_arr[rdata->aarx];
    rdata->beam_nb = beam_nb;
    rdata->frame_parms = &gNB->frame_parms;
    rdata->pusch_vars = &gNB->pusch_vars[ul_id];
    rdata->chest_freq = gNB->chest_freq;
    rdata->rxdataF = gNB->common_vars.rxdataF;
    rdata->ans = &ans[job_id];
    // Call the nr_pusch_antenna_processing function
    if (job_id == num_jobs - 1) {
        // Run the last job inline
        nr_pusch_antenna_processing(rdata);
    } else {
        pushTpool(&gNB->threadPool, task);
    }
    LOG_D(PHY, "Added Antenna (count %d/%d) to process, in pipe\n", job_id, num_jobs);
} // Antenna Loop

join_task_ans(ans, num_jobs - 1);

stop_meas(&gNB->pusch_channel_estimation_antenna_processing_stats);
for (int aarx = 0; aarx < gNB->frame_parms.nb_antennas_rx; aarx++) {
    *max_ch = max(*max_ch, max_ch_arr[aarx]);
    noise_amp2 += noise_amp2_arr[aarx];
    nest_count += nest_count_arr[aarx];
}
// get the maximum delay
*delay = delay_arr[0];
for (int aarx = 1; aarx < gNB->frame_parms.nb_antennas_rx; aarx++) {
    if (delay_arr[aarx].est_delay >= delay->est_delay) {

```

```

        *delay = delay_arr[aarx];
    }
}

#ifdef DEBUG_CH
    fclose(debug_ch_est);
#endif

    if (nvar && nest_count > 0) {
        *nvar = (uint32_t)(noise_amp2 / nest_count);
    }

    return 0;
}

/*****
 *
 * NAME :          nr_pusch_ptrs_processing
 *
 * PARAMETERS :    gNB          : gNB data structure
 *                  rel15_ul     : UL parameters
 *                  UE_id        : UE ID
 *                  nr_tti_rx     : slot rx TTI
 *                  dmrs_symbol_flag: DMRS Symbol Flag
 *                  symbol       : OFDM Symbol
 *                  nb_re_pusch  : PUSCH RE's
 *                  nb_re_pusch  : PUSCH RE's
 *
 * RETURN :        nothing
 *
 * DESCRIPTION :
 * If ptrs is enabled process the symbol accordingly
 * 1) Estimate phase noise per PTRS symbol
 * 2) Interpolate PTRS estimated value in TD after all PTRS symbols
 * 3) Compensated DMRS based estimated signal with PTRS estimation for slot
 *****/
// #define DEBUG_UL_PTRS
void nr_pusch_ptrs_processing(PHY_VARS_gNB *gNB,
                             NR_DL_FRAME_PARMS *frame_parms,
                             nfapi_nr_pusch_pdu_t *rel15_ul,
                             uint8_t ulsch_id,
                             uint8_t nr_tti_rx,
                             unsigned char symbol,
                             uint32_t nb_re_pusch)
{
    NR_gNB_PUSCH *pusch_vars = &gNB->pusch_vars[ulsch_id];
    int32_t *ptrs_re_symbol = NULL;
    int8_t ret = 0;
    uint8_t symbInSlot = rel15_ul->start_symbol_index + rel15_ul->nr_of_symbols;
    uint8_t *startSymbIndex = &rel15_ul->start_symbol_index;
    uint8_t *nbSymb = &rel15_ul->nr_of_symbols;
    uint8_t *L_ptr = &rel15_ul->pusch_ptrs.ptrs_time_density;

```

```

uint8_t *K_ptrs = &rel15_ul->pusch_ptrs.ptrs_freq_density;
uint16_t *dmrsSymbPos = &rel15_ul->ul_dmrs_symb_pos;
uint16_t *ptrsSymbPos = &pusch_vars->ptrs_symbols;
uint8_t *ptrsSymbIdx = &pusch_vars->ptrs_symbol_index;
uint16_t *nb_rb = &rel15_ul->rb_size;
uint8_t *ptrsReOffset = &rel15_ul->pusch_ptrs.ptrs_ports_list[0].ptrs_re_offset;

/* loop over antennas */
for (int aarx = 0; aarx < frame_parms->nb_antennas_rx; aarx++) {
    c16_t *phase_per_symbol = (c16_t *)pusch_vars->ptrs_phase_per_slot[aarx];
    ptrs_re_symbol = &pusch_vars->ptrs_re_per_slot;
    *ptrs_re_symbol = 0;
    phase_per_symbol[symbol].i = 0;
    /* set DMRS estimates to 0 angle with magnitude 1 */
    if (is_dmrs_symbol(symbol, *dmrsSymbPos)) {
        /* set DMRS real estimation to 32767 */
        phase_per_symbol[symbol].r = INT16_MAX; // 32767
#ifdef DEBUG_UL_PTRS
        printf("[PHY][PTRS]: DMRS Symbol %d -> %4d + j*%4d\n", symbol,
            phase_per_symbol[symbol].r, phase_per_symbol[symbol].i);
#endif
    } else { // real ptrs value is set to 0
        phase_per_symbol[symbol].r = 0;
    }

    if (symbol == *startSymbIndex) {
        *ptrsSymbPos = 0;
        set_ptr_symb_idx(ptrsSymbPos, *nbSymb, *startSymbIndex, 1 << *L_ptrs, *dmrsSymbPos);
    }

    /* if not PTRS symbol set current ptrs symbol index to zero*/
    *ptrsSymbIdx = 0;

    /* Check if current symbol contains PTRS */
    if (is_ptr_symb(symbol, *ptrsSymbPos)) {
        *ptrsSymbIdx = symbol;
    }

    /*-----
    ----- */
    /* 1) Estimate common phase error per PTRS symbol
    */

    /*-----
    ----- */
    const uint32_t *gold = nr_gold_pusch(frame_parms->N_RB_UL,
        frame_parms->symbols_per_slot,
        gNB->gNB_config.cell_config.phy_cell_id.value,
        rel15_ul->scid,
        nr_tti_rx,
        symbol);

    nr_ptr_s_cpe_estimation(*K_ptrs,
        *ptrsReOffset,

```

```

        *nb_rb,
        rel15_ul->rnti,
        nr_tti_rx,
        symbol,
        frame_parms->ofdm_symbol_size,
        (int16_t *)&pusch_vars->rxdataF_comp[aarx][(symbol *
nb_re_pusch)],

        gold,
        (int16_t *)&phase_per_symbol[symbol],
        ptrs_re_symbol);
    }

    /* For last OFDM symbol at each antenna perform interpolation and compensation for the
slot*/
    if (symbol == (symbInSlot - 1)) {

/*-----*/
/* 2) Interpolate PTRS estimated value in TD */

/*-----*/
/* If L-PTRS is > 0 then we need interpolation */
    if (*L_ptrs > 0) {
        ret = nr_ptrs_process_slot(*dmrsSymbPos, *ptrsSymbPos, (int16_t *)phase_per_symbol,
*startSymbIndex, *nbSymb);
        if (ret != 0) {
            LOG_W(PHY, "[PTRS] Compensation is skipped due to error in PTRS slot processing
!!\n");
        }
    }

/*-----*/
/* 3) Compensated DMRS based estimated signal with PTRS estimation
*/

/*-----*/
    for (uint8_t i = *startSymbIndex; i < symbInSlot; i++) {
        /* DMRS Symbol has 0 phase so no need to rotate the respective symbol */
        /* Skip rotation if the slot processing is wrong */
        if ((!is_dmrs_symbol(i, *dmrsSymbPos)) && (ret == 0)) {
#ifdef DEBUG_UL_PTRS
            printf("[PHY][UL][PTRS]: Rotate Symbol %2d with  %d + j* %d\n", i,
phase_per_symbol[i].r, phase_per_symbol[i].i);
#endif
            rotate_cpx_vector((c16_t *)&pusch_vars->rxdataF_comp[aarx][i * nb_re_pusch],
&phase_per_symbol[i],
(c16_t *)&pusch_vars->rxdataF_comp[aarx][i * nb_re_pusch],
((*nb_rb) * NR_NB_SC_PER_RB),

```

```

        15);
    } // if not DMRS Symbol
} // symbol loop
} // last symbol check
} // Antenna loop
}

int nr_srs_channel_estimation(
    const PHY_VARS_gNB *gNB,
    const int frame,
    const int slot,
    const nfapi_nr_srs_pdu_t *srs_pdu,
    const nr_srs_info_t *nr_srs_info,
    const c16_t **srs_generated_signal,
    int32_t srs_received_signal[][gNB->frame_parms.ofdm_symbol_size * (1 <<
srs_pdu->num_symbols)],
    int32_t srs_estimated_channel_freq[][1 << srs_pdu->num_ant_ports]
        [gNB->frame_parms.ofdm_symbol_size * (1 <<
srs_pdu->num_symbols)],
    int32_t srs_estimated_channel_time[][1 <<
srs_pdu->num_ant_ports][gNB->frame_parms.ofdm_symbol_size],
    int32_t srs_estimated_channel_time_shifted[][1 <<
srs_pdu->num_ant_ports][gNB->frame_parms.ofdm_symbol_size],
    int8_t *snr_per_rb,
    int8_t *snr)
{
#ifdef SRS_DEBUG
    if (srs_debug_fptr == NULL && first_time_flag) {
        srs_debug_fptr = get_new_logfile();
        previousTimestamp = current_time_sec();
        first_time_flag = 0;
        LOG_I(NR_PHY, "Opening srs_debug.log for the first time.");
    } else if (srs_debug_fptr != NULL && !first_time_flag) {
        if (current_time_sec() - previousTimestamp > 3600) {
            fclose(srs_debug_fptr);
            srs_debug_fptr = get_new_logfile();
            previousTimestamp = current_time_sec();
            LOG_I(NR_PHY, "Opening new srs_debug.log file.");
        }
    } else {
        if (srs_debug_fptr != NULL)
            fclose(srs_debug_fptr);
        LOG_I(NR_PHY, "ERROR: Couldn't open file srs_debug.log");
    }

    if (srs_debug_fptr != NULL) {
        print_formatted_time(srs_debug_fptr);
        fprintf(srs_debug_fptr, "Calling %s function\n", __FUNCTION__);
    }
    // LOG_I(NR_PHY, "Calling %s function\n", __FUNCTION__);
#endif
}

```

```

const NR_DL_FRAME_PARMS *frame_parms = &gNB->frame_parms;
const uint64_t subcarrier_offset = frame_parms->first_carrier_offset + srs_pdu->bwp_start
* NR_NB_SC_PER_RB;

const uint8_t N_ap = 1 << srs_pdu->num_ant_ports;
const uint8_t K_TC = 2 << srs_pdu->comb_size;
const uint16_t m_SRS_b = get_m_srs(srs_pdu->config_index, srs_pdu->bandwidth_index);
const uint16_t M_sc_b_SRS = m_SRS_b * NR_NB_SC_PER_RB / K_TC;
uint8_t fd_cdm = N_ap;
if (N_ap == 4 && ((K_TC == 2 && srs_pdu->cyclic_shift >= 4) || (K_TC == 4 &&
srs_pdu->cyclic_shift >= 6))) {
    fd_cdm = 2;
}

c16_t srs_ls_estimated_channel[frame_parms->ofdm_symbol_size * (1 <<
srs_pdu->num_symbols)];
uint32_t noise_power_per_rb[srs_pdu->bwp_size];

const uint32_t arr_len = frame_parms->nb_antennas_rx * N_ap * M_sc_b_SRS;

c16_t ch[arr_len];
memset(ch, 0, arr_len * sizeof(c16_t));

c16_t noise[arr_len];
memset(noise, 0, arr_len * sizeof(c16_t));

uint8_t mem_offset = ((16 - ((long)&srs_estimated_channel_freq[0][0][subcarrier_offset +
nr_srs_info->k_0_p[0][0]])) & 0xF)
>> 2; // >> 2 <=> /sizeof(int32_t)

// filt16_end is {4096,8192,8192,8192,12288,16384,16384,16384,0,0,0,0,0,0,0}
// The End of OFDM symbol corresponds to the position of last 16384 in the filter
// The c16multaddVectRealComplex applies the remaining 8 zeros of filter, therefore, to
avoid a buffer overflow,
// we added 8 in the array size
int32_t srs_est[frame_parms->ofdm_symbol_size * (1 << srs_pdu->num_symbols) + mem_offset +
8] __attribute__((aligned(32)));
c16_t ls_estimated = {0};

for (int ant = 0; ant < frame_parms->nb_antennas_rx; ant++) {
    for (int p_index = 0; p_index < N_ap; p_index++) {
        memset(srs_ls_estimated_channel, 0, frame_parms->ofdm_symbol_size * (1 <<
srs_pdu->num_symbols) * sizeof(c16_t));
        memset(srs_est, 0, (frame_parms->ofdm_symbol_size * (1 << srs_pdu->num_symbols) +
mem_offset) * sizeof(int32_t));

#ifdef SRS_DEBUG
        // LOG_I(NR_PHY, "===== UE port %d --> gNB Rx antenna %i
=====\\n", p_index, ant);
        if (srs_debug_fptr != NULL) {
            print_formatted_time(srs_debug_fptr);

```

```

        fprintf(srs_debug_fptr, "===== UE port %d --> gNB Rx antenna %i
=====\\n", p_index, ant);
    }
#endif

    uint16_t subcarrier = subcarrier_offset + nr_srs_info->k_0_p[p_index][0];
    if (subcarrier > frame_parms->ofdm_symbol_size) {
        subcarrier -= frame_parms->ofdm_symbol_size;
    }

    c16_t *srs_estimated_channel16 = (c16_t *)&srs_est[subcarrier + mem_offset];

    for (int k = 0; k < M_sc_b_SRS; k++) {
        if (k % fd_cdm == 0) {
            ls_estimated = (c16_t){0, 0};
            uint16_t subcarrier_cdm = subcarrier;

            for (int cdm_idx = 0; cdm_idx < fd_cdm; cdm_idx++) {
                int16_t generated_real = srs_generated_signal[p_index][subcarrier_cdm].r;
                int16_t generated_imag = srs_generated_signal[p_index][subcarrier_cdm].i;

                int16_t received_real = ((c16_t *)srs_received_signal[ant])[subcarrier_cdm].r;
                int16_t received_imag = ((c16_t *)srs_received_signal[ant])[subcarrier_cdm].i;

                // We know that nr_srs_info->srs_generated_signal_bits bits are enough to
                // represent the generated_real and
                // generated_imag. So we only need a nr_srs_info->srs_generated_signal_bits
                // shift to ensure that the result fits into 16
                // bits.
                ls_estimated.r += (int16_t)((((int32_t)generated_real * received_real +
                (int32_t)generated_imag * received_imag)
                >> nr_srs_info->srs_generated_signal_bits);
                ls_estimated.i += (int16_t)((((int32_t)generated_real * received_imag -
                (int32_t)generated_imag * received_real)
                >> nr_srs_info->srs_generated_signal_bits);

                // Subcarrier increment
                subcarrier_cdm += K_TC;
                if (subcarrier_cdm >= frame_parms->ofdm_symbol_size) {
                    subcarrier_cdm = subcarrier_cdm - frame_parms->ofdm_symbol_size;
                }
            }
        }
    }

    srs_ls_estimated_channel[subcarrier] = ls_estimated;

#ifdef SRS_DEBUG

    /*
    int subcarrier_log = subcarrier - subcarrier_offset;
    if (subcarrier_log < 0) {
        subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
    }
    */

```



```

    }
    if (subcarrier_log % 12 == 0) {
        LOG_I(NR_PHY, "----- %d
-----\n", subcarrier_log / 12);

        LOG_I(NR_PHY, "\t
__genRe__genIm__|__rxRe__rxIm__|__lsRe__lsIm__\n");
    }
    LOG_I(NR_PHY,
        "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
        subcarrier_log,
        ((c16_t *)srs_generated_signal[p_index])[subcarrier].r,
        ((c16_t *)srs_generated_signal[p_index])[subcarrier].i,
        ((c16_t *)srs_received_signal[ant])[subcarrier].r,
        ((c16_t *)srs_received_signal[ant])[subcarrier].i,
        ls_estimated.r,
        ls_estimated.i);
    */

    if (srs_debug_fptr != NULL) {
        int subcarrier_log = subcarrier - subcarrier_offset;
        if (subcarrier_log < 0) {
            subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
        }
        if (subcarrier_log % 12 == 0) {
            fprintf(srs_debug_fptr,
                "----- %d
-----\n",
                subcarrier_log / 12);
            fprintf(srs_debug_fptr, "\t
__genRe__genIm__|__rxRe__rxIm__|__lsRe__lsIm__\n");
        }
        fprintf(srs_debug_fptr,
            "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
            subcarrier_log,
            ((c16_t *)srs_generated_signal[p_index])[subcarrier].r,
            ((c16_t *)srs_generated_signal[p_index])[subcarrier].i,
            ((c16_t *)srs_received_signal[ant])[subcarrier].r,
            ((c16_t *)srs_received_signal[ant])[subcarrier].i,
            ls_estimated.r,
            ls_estimated.i);
    }
#endif

    const uint16_t sc_offset = subcarrier + mem_offset;

    // Channel interpolation
    if (srs_pdu->comb_size == 0) {
        if (k == 0) { // First subcarrier case
            // filt8_start is {12288,8192,4096,0,0,0,0,0}
            c16multaddVectRealComplex(filt8_start, &ls_estimated, srs_estimated_channel16,
8);

            } else if (subcarrier < K_TC) { // Start of OFDM symbol case

```

```

        // filt8_start is {12288,8192,4096,0,0,0,0,0}
        srs_estimated_channel16 = (c16_t *)&srs_est[subcarrier];
        const short *filter = mem_offset == 0 ? filt8_start : filt8_start_shift2;
        c16multaddVectRealComplex(filter, &ls_estimated, srs_estimated_channel16, 8);
    } else if ((subcarrier + K_TC) >= frame_parms->ofdm_symbol_size
        || k == (M_sc_b_SRS - 1)) { // End of OFDM symbol or last subcarrier
cases
        // filt8_end is {4096,8192,12288,16384,0,0,0,0}
        const short *filter = mem_offset == 0 || k == (M_sc_b_SRS - 1) ? filt8_end :
filt8_end_shift2;
        c16multaddVectRealComplex(filter, &ls_estimated, srs_estimated_channel16, 8);
    } else if (k % 2 == 1) { // 1st middle case
        // filt8_middle2 is {4096,8192,8192,8192,4096,0,0,0}
        c16multaddVectRealComplex(filt8_middle2, &ls_estimated, srs_estimated_channel16,
8);
    } else if (k % 2 == 0) { // 2nd middle case
        // filt8_middle4 is {0,0,4096,8192,8192,8192,4096,0}
        c16multaddVectRealComplex(filt8_middle4, &ls_estimated, srs_estimated_channel16,
8);
        srs_estimated_channel16 = (c16_t *)&srs_est[sc_offset];
    }
} else {
    if (k == 0) { // First subcarrier case
        // filt16_start is {12288,8192,8192,8192,4096,0,0,0,0,0,0,0,0,0,0,0}
        c16multaddVectRealComplex(filt16_start, &ls_estimated, srs_estimated_channel16,
16);
    } else if (subcarrier < K_TC) { // Start of OFDM symbol case
        srs_estimated_channel16 = (c16_t *)&srs_est[sc_offset];
        // filt16_start is {12288,8192,8192,8192,4096,0,0,0,0,0,0,0,0,0,0,0}
        c16multaddVectRealComplex(filt16_start, &ls_estimated, srs_estimated_channel16,
16);
    } else if ((subcarrier + K_TC) >= frame_parms->ofdm_symbol_size
        || k == (M_sc_b_SRS - 1)) { // End of OFDM symbol or last subcarrier
cases
        // filt16_end is {4096,8192,8192,8192,12288,16384,16384,16384,0,0,0,0,0,0,0,0}
        c16multaddVectRealComplex(filt16_end, &ls_estimated, srs_estimated_channel16,
16);
    } else { // Middle case
        // filt16_middle4 is
{4096,8192,8192,8192,8192,8192,8192,8192,4096,0,0,0,0,0,0,0}
        c16multaddVectRealComplex(filt16_middle4, &ls_estimated,
srs_estimated_channel16, 16);
        srs_estimated_channel16 = (c16_t *)&srs_est[sc_offset];
    }
}

// Subcarrier increment
subcarrier += K_TC;
if (subcarrier >= frame_parms->ofdm_symbol_size) {
    subcarrier = subcarrier - frame_parms->ofdm_symbol_size;
}

```

```

    } // for (int k = 0; k < M_sc_b_SRS; k++)

    memcpy(&srs_estimated_channel_freq[ant][p_index][0],
           &srs_est[mem_offset],
           (frame_parms->ofdm_symbol_size * (1 << srs_pdu->num_symbols)) *
sizeof(int32_t));

    // Compute noise
    subcarrier = subcarrier_offset + nr_srs_info->k_0_p[p_index][0];
    if (subcarrier > frame_parms->ofdm_symbol_size) {
        subcarrier -= frame_parms->ofdm_symbol_size;
    }
    uint16_t base_idx = ant * N_ap * M_sc_b_SRS + p_index * M_sc_b_SRS;
    for (int k = 0; k < M_sc_b_SRS; k++) {
        ch[base_idx + k] = ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier];
        noise[base_idx + k].r = abs(srs_ls_estimated_channel[subcarrier].r - ch[base_idx +
k].r);
        noise[base_idx + k].i = abs(srs_ls_estimated_channel[subcarrier].i - ch[base_idx +
k].i);
        subcarrier += K_TC;
        if (subcarrier >= frame_parms->ofdm_symbol_size) {
            subcarrier = subcarrier - frame_parms->ofdm_symbol_size;
        }
    }

#ifdef SRS_DEBUG
    /*
    subcarrier = subcarrier_offset + nr_srs_info->k_0_p[p_index][0];
    if (subcarrier > frame_parms->ofdm_symbol_size) {
        subcarrier -= frame_parms->ofdm_symbol_size;
    }

    for (int k = 0; k < K_TC * M_sc_b_SRS; k++) {
        int subcarrier_log = subcarrier - subcarrier_offset;
        if (subcarrier_log < 0) {
            subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
        }

        if (subcarrier_log % 12 == 0) {
            LOG_I(NR_PHY, "----- %d
-----\n", subcarrier_log / 12);
            LOG_I(NR_PHY, "\t
__lsRe__lsIm__|__intRe__intIm__|__noiRe__noiIm__\n");
        }

        LOG_I(NR_PHY,
              "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
              subcarrier_log,
              srs_ls_estimated_channel[subcarrier].r,
              srs_ls_estimated_channel[subcarrier].i,
              ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier].r,
              ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier].i,

```

```

        noise[base_idx + (k / K_TC)].r,
        noise[base_idx + (k / K_TC)].i);

    // Subcarrier increment
    subcarrier++;
    if (subcarrier >= frame_parms->ofdm_symbol_size) {
        subcarrier = subcarrier - frame_parms->ofdm_symbol_size;
    }
}
*/

if (srs_debug_fptr != NULL) {
    subcarrier = subcarrier_offset + nr_srs_info->k_0_p[p_index][0];
    if (subcarrier > frame_parms->ofdm_symbol_size) {
        subcarrier -= frame_parms->ofdm_symbol_size;
    }

    for (int k = 0; k < K_TC * M_sc_b_SRS; k++) {
        int subcarrier_log = subcarrier - subcarrier_offset;
        if (subcarrier_log < 0) {
            subcarrier_log = subcarrier_log + frame_parms->ofdm_symbol_size;
        }

        if (subcarrier_log % 12 == 0) {
            fprintf(srs_debug_fptr,
                    "----- %d\n",
                    subcarrier_log / 12);
            fprintf(srs_debug_fptr, "\t
__lsRe__ __lsIm__ | __intRe__ __intIm__ | __noiRe__ __noiIm__ \n");
        }

        fprintf(srs_debug_fptr,
                "(%4i) %6i\t%6i | %6i\t%6i | %6i\t%6i\n",
                subcarrier_log,
                srs_ls_estimated_channel[subcarrier].r,
                srs_ls_estimated_channel[subcarrier].i,
                ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier].r,
                ((c16_t *)srs_estimated_channel_freq[ant][p_index])[subcarrier].i,
                noise[base_idx + (k / K_TC)].r,
                noise[base_idx + (k / K_TC)].i);

        // Subcarrier increment
        subcarrier++;
        if (subcarrier >= frame_parms->ofdm_symbol_size) {
            subcarrier = subcarrier - frame_parms->ofdm_symbol_size;
        }
    }
}
#endif

// Convert to time domain

```

```

        freq2time(gNB->frame_parms.ofdm_symbol_size,
                  (int16_t *)srs_estimated_channel_freq[ant][p_index],
                  (int16_t *)srs_estimated_channel_time[ant][p_index]);

        memcpy(&srs_estimated_channel_time_shifted[ant][p_index][0],
               &srs_estimated_channel_time[ant][p_index][gNB->frame_parms.ofdm_symbol_size >>
1],
               (gNB->frame_parms.ofdm_symbol_size >> 1) * sizeof(int32_t));

memcpy(&srs_estimated_channel_time_shifted[ant][p_index][gNB->frame_parms.ofdm_symbol_size
>> 1],
        &srs_estimated_channel_time[ant][p_index][0],
        (gNB->frame_parms.ofdm_symbol_size >> 1) * sizeof(int32_t));

    } // for (int p_index = 0; p_index < N_ap; p_index++)
} // for (int ant = 0; ant < frame_parms->nb_antennas_rx; ant++)

// Compute signal power
uint32_t signal_power = max(signal_energy_nodc(ch, arr_len), 1);

#ifdef SRS_DEBUG
    // LOG_I(NR_PHY, "signal_power = %u\n", signal_power);

    if (srs_debug_fptr != NULL) {
        fprintf(srs_debug_fptr, "signal_power = %u\n", signal_power);
    }
#endif

if (signal_power == 0) {
    LOG_W(NR_PHY, "Received SRS signal power is 0\n");
    return -1;
}

// Compute noise power
const uint8_t srs_symbols_per_rb = srs_pdu->comb_size == 0 ? 6 : 3;
const uint8_t n_noise_est = frame_parms->nb_antennas_rx * N_ap * srs_symbols_per_rb;
uint64_t sum_re = 0;
uint64_t sum_re2 = 0;
uint64_t sum_im = 0;
uint64_t sum_im2 = 0;

for (int rb = 0; rb < m_SRS_b; rb++) {
    sum_re = 0;
    sum_re2 = 0;
    sum_im = 0;
    sum_im2 = 0;

    for (int ant = 0; ant < frame_parms->nb_antennas_rx; ant++) {
        for (int p_index = 0; p_index < N_ap; p_index++) {
            uint16_t base_idx = ant * N_ap * M_sc_b_SRS + p_index * M_sc_b_SRS + rb *
srs_symbols_per_rb;

```

```

        for (int srs_symb = 0; srs_symb < srs_symbols_per_rb; srs_symb++) {
            sum_re = sum_re + noise[base_idx + srs_symb].r;
            sum_re2 = sum_re2 + noise[base_idx + srs_symb].r * noise[base_idx + srs_symb].r;
            sum_im = sum_im + noise[base_idx + srs_symb].i;
            sum_im2 = sum_im2 + noise[base_idx + srs_symb].i * noise[base_idx + srs_symb].i;
        } // for (int srs_symb = 0; srs_symb < srs_symbols_per_rb; srs_symb++)
    } // for (int p_index = 0; p_index < N_ap; p_index++)
} // for (int ant = 0; ant < frame_parms->nb_antennas_rx; ant++)

    noise_power_per_rb[rb] = max(sum_re2 / n_noise_est - (sum_re / n_noise_est) * (sum_re /
n_noise_est) + sum_im2 / n_noise_est
                                - (sum_im / n_noise_est) * (sum_im / n_noise_est),
                                1);
    snr_per_rb[rb] = dB_fixed(signal_power) - dB_fixed(noise_power_per_rb[rb]);

#ifdef SRS_DEBUG
    // LOG_I(NR_PHY, "noise_power_per_rb[%i] = %i, snr_per_rb[%i] = %i dB\n", rb,
noise_power_per_rb[rb], rb, snr_per_rb[rb]);

    if (srs_debug_fptr != NULL) {
        fprintf(srs_debug_fptr,
                "noise_power_per_rb[%i] = %i, snr_per_rb[%i] = %i dB\n",
                rb,
                noise_power_per_rb[rb],
                rb,
                snr_per_rb[rb]);
    }
#endif

} // for (int rb = 0; rb < m_SRS_b; rb++)

const uint32_t noise_power = max(signal_energy_nodc(noise, arr_len), 1);

*snr = dB_fixed(signal_power) - dB_fixed(noise_power);

#ifdef SRS_DEBUG
    // LOG_I(NR_PHY, "noise_power = %u, SNR = %i dB\n", noise_power, *snr);

    if (srs_debug_fptr != NULL) {
        fprintf(srs_debug_fptr, "noise_power = %u, SNR = %i dB\n", noise_power, *snr);
    }
#endif

    return 0;
}

```

2. Enabling the OAIBOX CSI Data Reporting

To enable the OAIBOX CSI Data Reporting, in the file
/home/user/openairinterface5g/openair1/PHY/NR_ESTIMATION/nr_ul_channel_estimation.c
enable the flag SRS_DEBUG by uncommenting its preprocessor define directive, as shown
below.

```
18 #include "PHY/NR_UE_TRANSPORT/srs_modulation_nr.h"
17 #include "PHY/NR_UE_ESTIMATION/filt16a_32.h"
16 #include "PHY/NR_TRANSPORT/nr_sch_dmrs.h"
15 #include "PHY/NR_REFSIG/ul_ref_seq_nr.h"
14 #include "executables/softmodem-common.h"
13 #include "nr_phy_common.h"
12
11 // custom added headers
10 #include <stdio.h>
9 #include <time.h>
8 #include "custom_time_functions_utils.c"
7
6 FILE *srs_debug_fptr = NULL;
5 int first_time_flag = 1;
4 long long previousTimestamp;
3
2 // #define DEBUG_CH
1 // #define DEBUG_PUSCH
48 #define SRS_DEBUG
3
2 #define NO_INTERP 1
1 #define dBc(x, y) (dB_fixed(((int32_t)(x)) * (x) + ((int32_t)(y)) * (y)))
4
5 __attribute__((always_inline)) inline c16_t c32x16cumulVectVectWithSteps(c16_t *in1, // clang: Unknown type name 'c16_t'
6 int *offset1,
7 const int step1,
8 c16_t *in2, // clang: Unknown type name 'c16_t'
9 )
10 {
11     // ...
12 }
```

After doing so, save the file and make sure the 5G Core Network and gNB are turned off, then compile the new code either by running the following 2 commands or by using the compile button (I think, never tried it though) in the OAIBOX Dashboard.

```
cd /home/user/openairinterface5g/cmake_targets
./build_oai -w USRP --ninja --gNB -c
```

Overview

Mobility

Raycap

OAIBOX 40

OAIBOX 40

Online

Client Id

Machine Id

OAIBOX-2-1220100000-00254

Machine Serial Number

NI B200

Software Defined Radio Device

Serial: 345756A

2025/08/15 20:24:56

Last update

Reboot

CN5G

Stopped

Start

gNB

Stopped

OAIBOX_2024.46.1

Start

Configuration

Logs

Reset

Compile

Network Status

1 gNBs

0 UEs

0 bps Downlink

0 bps Uplink

Network Overview

User Equipments

IMSI

0000000009

RSSI

-77.1 dBm

RSRP

-97 dBm

After the compilation is complete, turn the 5G Core Network and gNB back on and connect your UE to the network. The CSI log data is recorded in the directory `/home/user/openairinterface5g/cmake_targets/ran_build/build` as files with the name `srs_debug_<timestamp>.log`, having the `<timestamp>` follow the format `yy:mm:dd:HH:MM` (for example, a file `srs_debug_25_08_12_16_22.log` would have been recorded on 12.08.2025 at 16:22). Keep in mind that the size of the log files increases very quickly, at a rate of approximately 2 MB/s.

To stop the logging process, simply stop the gNB and 5G Core Network, comment back the `SRS_DEBUG` flag in the file `/home/user/openairinterface5g/openair1/PHY/NR_ESTIMATION/nr_ul_channel_estimation.c` and then recompile the code like before.

The log files generated by the OAIBOX will have the following format.

To parse the log files into newline-delimited JSON files (NDJSON files), the following python script was used.

```
#!/usr/bin/env python3
import argparse
import json
import os
import re
from typing import Dict, Tuple, Optional

# -----
# Regex patterns
# -----
TS_RE = re.compile(r'^Timestamp:\s+(.+?)\s*$')
CALL_RE = re.compile(r'^Calling\s+nr_srs_channel_estimation function')
UE_HDR_RE = re.compile(r'^UE port\s+(\d+)\s*-->\s*gNB Rx antenna\s+(\d+)')
BLOCK_SEP_RE = re.compile(r'^-\s+(\d+)\s+-+\s*$')
GRID1_HDR_RE = re.compile(r'^__genRe') # gen/rx/ls grid
GRID2_HDR_RE = re.compile(r'^__lsRe') # ls/int/noi grid
ROW6_RE = re.compile(
    r'^\s*(\d+)\s*' # ( index )
    r'([+-]?\d+)\s+([+-]?\d+)\s*|\s*'
    r'([+-]?\d+)\s+([+-]?\d+)\s*|\s*'
    r'([+-]?\d+)\s+([+-]?\d+)\s*$'
)
SIGNAL_POWER_RE = re.compile(r'^signal_power\s*=\s*(\d+)\s*$')
NOISE_RB_RE = re.compile(
    r'^noise_power_per_rb\[(\d+)\]\s*=\s*(\d+),\s*snr_per_rb\[(\d+)\]\s*=\s*(\d+)\s*dB\s*$'
)
NOISE_POWER_RE = re.compile(r'^noise_power\s*=\s*(\d+),\s*SNR\s*=\s*(\d+)\s*dB\s*$')

# -----
# JSON sink that supports 3 modes
# -----
class JsonSink:
    def __init__(self, outdir: str, mode: str):
        self.mode = mode
        os.makedirs(outdir, exist_ok=True)
        self._first = True
        self._fh = None
        self._dir = outdir
        self._rec_dir = None
        if mode == 'ndjson':
            self._fh = open(os.path.join(outdir, 'records.ndjson'), 'w', encoding='utf-8')
        elif mode == 'array':
            self._fh = open(os.path.join(outdir, 'records.json'), 'w', encoding='utf-8')
            self._fh.write('[\n')
        elif mode == 'per-record':
            self._rec_dir = os.path.join(outdir, 'records')
```

```

        os.makedirs(self._rec_dir, exist_ok=True)
    else:
        raise ValueError("mode must be one of: ndjson, array, per-record")

    @staticmethod
    def _sanitize_filename(s: str) -> str:
        # Keep digits, letters, and selected safe punctuation
        return re.sub(r'^A-Za-z0-9_-.+ ', '_', s)

    def write_record(self, obj: dict, record_id_hint: Optional[str] = None):
        if self.mode == 'ndjson':
            self._fh.write(json.dumps(obj, separators=(',', ':')) + '\n')
        elif self.mode == 'array':
            if not self._first:
                self._fh.write(',\n')
            self._fh.write(json.dumps(obj, separators=(',', ':')))
            self._first = False
        else: # per-record
            if record_id_hint:
                fname = f"record_{self._sanitize_filename(record_id_hint)}.json"
            else:
                fname = "record.json"
            path = os.path.join(self._rec_dir, fname)
            # Avoid overwriting if timestamps repeat
            base, ext = os.path.splitext(path)
            i = 1
            final = path
            while os.path.exists(final):
                final = f"{base}_{i}{ext}"
                i += 1
            with open(final, 'w', encoding='utf-8') as fh:
                json.dump(obj, fh, separators=(',', ':'))

    def close(self):
        if self.mode == 'array' and self._fh:
            self._fh.write('\n\n')
        if self._fh:
            self._fh.flush()
            self._fh.close()

# -----
# Record builder
# -----
class RecordBuilder:
    def __init__(self, timestamp: Optional[str]):
        self.timestamp = timestamp
        # sections[(port,ant)] = {'port': int, 'antenna': int, 'rb': {'rb_index': {...}}}
        self.sections: Dict[Tuple[int, int], dict] = {}
        self.metrics = {
            'signal_power': None,
            'per_rb': [], # list of {'rb': i, 'noise_power': n, 'snr_db': s}

```

```

        'noise_power': None,
        'snr_db': None
    }

def ensure_section(self, port: int, ant: int) -> dict:
    key = (port, ant)
    s = self.sections.get(key)
    if s is None:
        s = {'port': port, 'antenna': ant, 'rb': {}}
        self.sections[key] = s
    return s

@staticmethod
def ensure_rb(section: dict, rb_index: int) -> dict:
    rb_map = section['rb']
    rb = rb_map.get(rb_index)
    if rb is None:
        rb = {
            'rb_index': rb_index,
            'genrxls': [],
            'lsintnoi': []
        }
        rb_map[rb_index] = rb
    return rb

def to_jsonable(self) -> dict:
    # Convert sections dict to list, and rb maps to sorted lists
    sections_list = []
    for (_k), s in self.sections.items():
        rb_entries = list(s['rb'].values())
        rb_entries.sort(key=lambda x: x['rb_index'])
        sections_list.append({
            'port': s['port'],
            'antenna': s['antenna'],
            'rb': rb_entries
        })
    # Sort sections for stability
    sections_list.sort(key=lambda x: (x['port'], x['antenna']))
    return {
        'timestamp': self.timestamp,
        'sections': sections_list,
        'metrics': self.metrics
    }

# -----
# Parsing logic
# -----
def parse_to_json(input_path: str, outdir: str, mode: str = 'ndjson'):
    sink = JsonSink(outdir, mode)

    # State across lines

```

```

last_ts: Optional[str] = None
current: Optional[RecordBuilder] = None
current_port: Optional[int] = None
current_ant: Optional[int] = None
current_block: Optional[int] = None
current_grid: Optional[str] = None # 'genrxls' or 'lsintnoi'

def finish_record():
    nonlocal current, current_port, current_ant, current_block, current_grid
    if current is not None:
        obj = current.to_jsonable()
        sink.write_record(obj, record_id_hint=current.timestamp or "no_ts")
    # Reset record-scoped state
    current = None
    current_port = None
    current_ant = None
    current_block = None
    current_grid = None

with open(input_path, 'r', encoding='utf-8', errors='replace') as fh:
    for raw_line in fh:
        line = raw_line.rstrip('\n')

        # Timestamp lines
        m = TS_RE.match(line)
        if m:
            last_ts = m.group(1).strip()
            continue

        # Start of computation
        if CALL_RE.match(line):
            # Close any previous incomplete record gracefully
            finish_record()
            # Start new one
            current = RecordBuilder(timestamp=last_ts)
            continue

        # Section header: UE port -> gNB antenna
        m = UE_HDR_RE.search(line)
        if m:
            current_port = int(m.group(1))
            current_ant = int(m.group(2))
            if current is None:
                # In case logs miss the CALL line, start a record with last timestamp
                current = RecordBuilder(timestamp=last_ts)
            # Ensure section exists
            current.ensure_section(current_port, current_ant)
            continue

        # Grid headers
        if GRID1_HDR_RE.search(line):
            current_grid = 'genrxls'

```

```

        continue
    if GRID2_HDR_RE.search(line):
        current_grid = 'lsintnoi'
        continue

    # RB separator
    m = BLOCK_SEP_RE.match(line)
    if m:
        current_block = int(m.group(1))
        continue

    # Data rows: 3 pairs
    m = ROW6_RE.match(line)
    if m and current is not None and current_port is not None and current_ant is not
None and current_block is not None and current_grid:
        bin_idx = int(m.group(1))
        a1, a2, b1, b2, c1, c2 = map(int, m.groups()[1:])
        section = current.ensure_section(current_port, current_ant)
        rb = current.ensure_rb(section, current_block)

        if current_grid == 'genrxls':
            rb['genrxls'].append({
                'bin': bin_idx,
                'gen': {'re': a1, 'im': a2},
                'rx': {'re': b1, 'im': b2},
                'ls': {'re': c1, 'im': c2},
            })
        else: # lsintnoi
            rb['lsintnoi'].append({
                'bin': bin_idx,
                'ls': {'re': a1, 'im': a2},
                'interp': {'re': b1, 'im': b2},
                'noise': {'re': c1, 'im': c2},
            })
        continue

    # Metrics
    if current is not None:
        m = SIGNAL_POWER_RE.match(line)
        if m:
            current.metrics['signal_power'] = int(m.group(1))
            continue

        m = NOISE_RB_RE.match(line)
        if m:
            rb_index_1 = int(m.group(1))
            noise_power = int(m.group(2))
            rb_index_2 = int(m.group(3))
            snr_db = int(m.group(4))
            # Trust the first index; log shows both equal
            current.metrics['per_rb'].append({
                'rb': rb_index_1,

```

```

        'noise_power': noise_power,
        'snr_db': snr_db
    })
    continue

    m = NOISE_POWER_RE.match(line)
    if m:
        current.metrics['noise_power'] = int(m.group(1))
        current.metrics['snr_db'] = int(m.group(2))
        # End of record
        finish_record()
        continue

    # End-of-file: flush any partial record
    if current is not None:
        finish_record()

sink.close()

def main():
    ap = argparse.ArgumentParser(description="Parse SRS logs into JSON (streaming,
record-by-record).")
    ap.add_argument('input', help='Path to the large text log')
    ap.add_argument('--outdir', default='out', help='Output directory (default: ./out)')
    ap.add_argument('--mode', choices=['ndjson', 'array', 'per-record'], default='ndjson',
                    help='ndjson: records.ndjson (default); array: single records.json
array; '
                        'per-record: one JSON file per record')
    args = ap.parse_args()
    parse_to_json(args.input, args.outdir, args.mode)

if __name__ == '__main__':
    main()

```

The resulting NDJSON files will have objects with the following structure.

```

{
  "timestamp": "25:08:13:20:02:12:459983",
  "sections": [
    {
      "port": 0,
      "antenna": 0,
      "rb": [
        {
          "rb_index": 0,
          "genrxls": [
            {
              "bin": 0,

```

```
    "gen": {
      "re": 511,
      "im": 0
    },
    "rx": {
      "re": 167,
      "im": 393
    },
    "ls": {
      "re": 166,
      "im": 392
    }
  },
  {
    "bin": 2,
    "gen": {
      "re": 511,
      "im": 0
    },
    "rx": {
      "re": 167,
      "im": 393
    },
    "ls": {
      "re": 166,
      "im": 392
    }
  }
],
"lsintnoi": [
  {
    "bin": 0,
    "ls": {
      "re": 166,
      "im": 392
    },
    "interp": {
      "re": 158,
      "im": 392
    },
    "noise": {
      "re": 8,
      "im": 0
    }
  },
  {
    "bin": 1,
    "ls": {
      "re": 0,
      "im": 0
    },
    "interp": {
```



```

        "re": 150,
        "im": 390
    },
    "noise": {
        "re": 8,
        "im": 0
    }
}
]
},
{
    "rb_index": 0,
    "genrxls": [
        {
            "bin": 0,
            "gen": {
                "re": 511,
                "im": 0
            },
            "rx": {
                "re": 167,
                "im": 393
            },
            "ls": {
                "re": 166,
                "im": 392
            }
        },
        {
            "bin": 2,
            "gen": {
                "re": 511,
                "im": 0
            },
            "rx": {
                "re": 167,
                "im": 393
            },
            "ls": {
                "re": 166,
                "im": 392
            }
        }
    ],
    "lsintnoi": [
        {
            "bin": 0,
            "ls": {
                "re": 166,
                "im": 392
            },
            "interp": {

```

```

        "re": 158,
        "im": 392
    },
    "noise": {
        "re": 8,
        "im": 0
    }
},
{
    "bin": 1,
    "ls": {
        "re": 0,
        "im": 0
    },
    "interp": {
        "re": 150,
        "im": 390
    },
    "noise": {
        "re": 8,
        "im": 0
    }
}
]
}
]
},
{
    "port": 1,
    "antenna": 0,
    "rb": [
        {
            "rb_index": 0,
            "genrxls": [
                {
                    "bin": 0,
                    "gen": {
                        "re": 511,
                        "im": 0
                    },
                    "rx": {
                        "re": 167,
                        "im": 393
                    },
                    "ls": {
                        "re": 166,
                        "im": 392
                    }
                },
                {
                    "bin": 2,
                    "gen": {

```

```

        "re": 511,
        "im": 0
    },
    "rx": {
        "re": 167,
        "im": 393
    },
    "ls": {
        "re": 166,
        "im": 392
    }
}
],
"lsintnoi": [
    {
        "bin": 0,
        "ls": {
            "re": 166,
            "im": 392
        },
        "interp": {
            "re": 158,
            "im": 392
        },
        "noise": {
            "re": 8,
            "im": 0
        }
    },
    {
        "bin": 1,
        "ls": {
            "re": 0,
            "im": 0
        },
        "interp": {
            "re": 150,
            "im": 390
        },
        "noise": {
            "re": 8,
            "im": 0
        }
    }
]
},
{
    "rb_index": 0,
    "genrxls": [
        {
            "bin": 0,
            "gen": {

```

```

        "re": 511,
        "im": 0
    },
    "rx": {
        "re": 167,
        "im": 393
    },
    "ls": {
        "re": 166,
        "im": 392
    }
},
{
    "bin": 2,
    "gen": {
        "re": 511,
        "im": 0
    },
    "rx": {
        "re": 167,
        "im": 393
    },
    "ls": {
        "re": 166,
        "im": 392
    }
}
],
"lsintnoi": [
    {
        "bin": 0,
        "ls": {
            "re": 166,
            "im": 392
        },
        "interp": {
            "re": 158,
            "im": 392
        },
        "noise": {
            "re": 8,
            "im": 0
        }
    },
    {
        "bin": 1,
        "ls": {
            "re": 0,
            "im": 0
        },
        "interp": {
            "re": 150,

```

```
        "im": 390
      },
      "noise": {
        "re": 8,
        "im": 0
      }
    ]
  }
],
"metrics": {
  "signal_power": 12345,
  "per_rb": [
    {
      "rb": 0,
      "noise_power": 43,
      "snr_db": 32
    },
    {
      "rb": 1,
      "noise_power": 33,
      "snr_db": 33
    },
    {
      "rb": 2,
      "noise_power": 87,
      "snr_db": 29
    }
  ]
},
"noise_power": 299,
"snr_db": 24
}
```

4. Visualizing the OAIBOX CSI Data

To visualize the OAIBOX CSI Data, the aforementioned NDJSON was flattened into a CSV file using the following Python script.

```
import json
import csv
import sys
from datetime import datetime

def timestamp_to_microseconds(ts):
    """Convert YY:MM:DD:HH:MM:SS:uS to microseconds since beginning."""
    parts = ts.split(':')
    if len(parts) != 7:
        raise ValueError(f"Unexpected timestamp format: {ts}")

    year, month, day, hour, minute, second, microsec = map(int, parts)
    dt = datetime(year=2000+year, month=month, day=day, hour=hour, minute=minute,
second=second)
    total_microseconds = int(dt.timestamp() * 1_000_000) + microsec
    return total_microseconds

# input_file = r"out/srs_debug_25_08_13_20_02.ndjson"

if len(sys.argv) < 2:
    print("Usage: python.exe extract_CSI_data.py <input_file_path.ndjson>")
    sys.exit(1)

input_file = sys.argv[1]
output_file = r"out/srs_debug_25_08_13_20_02.csv"

start_time_us = None

with open(output_file, "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["timestamp", "bin", "ls_re", "ls_im"])

    with open(input_file, "r") as f:
        for line_num, line in enumerate(f, start=1):
            line = line.strip()
            if not line:
                continue # skip blank lines if they exist
```

```

try:
    obj = json.loads(line)
except:
    print(f"Error decoding line {line_num}: {e}")
    continue

try:
    timestamp_abs_us = timestamp_to_microseconds(obj["timestamp"])
except ValueError as e:
    print(f"Skipping line {line_num}: {e}")
    continue

if start_time_us is None:
    start_time_us = timestamp_abs_us

timestamp_rel_us = timestamp_abs_us - start_time_us

for section in obj["sections"]:
    for rb in section["rb"]:
        for genrxls in rb["genrxls"]:
            bin_val = genrxls["bin"]
            ls_re = genrxls["ls"]["re"]
            ls_im = genrxls["ls"]["im"]
            writer.writerow([timestamp_rel_us, bin_val, ls_re, ls_im])

print(f"Extraction complete. Data saved to {output_file}")

```

After flattening, the CSI Data was visualized as a spectrogram using the following MATLAB script.

```

close all
clear
clc
%% Read data from CSV
data = readtable('..\out\CSI_srs_debug_25_08_12_16_22.csv');
CSI = data.ls_re + 1j * data.ls_im;
time = data.timestamp / 1e6;
unique_time = unique(time);
freq_bin = data.bin;
unique_freq_bin = unique(freq_bin);
CSI_matrix = NaN(length(unique_time), length(unique_freq_bin));
for i = 1:height(data)
    t_idx = find(unique_time == time(i));
    f_idx = find(unique_freq_bin == freq_bin(i));

```

```

        CSI_matrix(t_idx, f_idx) = CSI(i);
    end
    %% Plotting data
    figure, imagesc(unique_freq_bin, unique_time, abs(CSI_matrix));
    set(gca, 'YDir', 'normal');
    xlabel('Bin'), ylabel('Time (s)'), title('CSI over time and freq bins');
    colorbar;
    %% Plot data with clipping at 750
    figure, imagesc(unique_freq_bin, unique_time, min(abs(CSI_matrix), 750));
    set(gca, 'YDir', 'normal');
    xlabel('Bin'), ylabel('Time (s)'), title('CSI (clipped) over time and freq bins');
    colorbar;

```

5. Changing the CSI slow-time sampling frequency:

To change the slow-time sampling frequency, you have to change the value of MAX_MOBILES_PER_GNB in the file
 /home/user/openairinterface5g/common/openairinterface5g_limits.h

The value of MAX_MOBILES_PER_GNB is proportional to the slow-time sampling frequency of the CSI data.

Further working points:

- ☐ Create range-doppler plots from the CSI data
- ☐ Create a heatmap detection plot from the CSI data
- ☐ Train AI model using YOLO/??? to detect ...